

Detailed Image Description Agent – Tool Installation and Logic

Overview

This agent is designed to create extremely detailed descriptions of arbitrary images by combining several computer-vision modules. Rather than relying solely on a vision-language model (VLM), the agent runs a pipeline of specialised tools and fuses their outputs into a rich textual description. The general steps are:

1. **Use a VLM** to produce a short initial caption. This gives context and helps the agent decide which modules are likely relevant.
2. **Run a high-level object detector (YOLO)** on every image to determine which objects are present ¹.
3. **Dynamically choose specialised tools** based on the detected objects (pose estimation, face landmarks, garment classification, segmentation, colour extraction, OCR, scene classification).
4. **Merge all extracted information** and feed it to a large language model to generate a comprehensive, structured description.

Required Tools and Installation

Tool / library	Purpose	Installation commands
YOLO (Ultralytics YOLOv5/YOLOv8)	Real-time object detection; predicts bounding boxes and class probabilities in a single pass ¹ ² .	<code>pip install ultralytics</code>
MediaPipe Pose Landmarker	Estimates 33 three-dimensional body landmarks (nose, shoulders, elbows, wrists, hips, knees, etc.) with normalized and world coordinates ³ .	<code>pip install mediapipe</code> (Pose is part of the mediapipe package)
MediaPipe Face Landmarker / Face Mesh	Predicts 468 3-D face landmarks in real time ⁴ .	Included in <code>mediapipe</code>
MediaPipe Selfie Segmentation & HairSegmenter	Segments hair, skin, clothes and accessories; hair segmenter returns a binary mask for hair ⁵ .	Included in <code>mediapipe</code>

Tool / library	Purpose	Installation commands
DeepFashion-based garment classifier	Recognizes clothing categories and style attributes (dress, pants, jacket, colour, pattern etc.), using a model trained on the DeepFashion dataset ⁶ .	Use a pre-trained model from e.g. <code>deepfashion-category-classifier</code> ; install dependencies with <code>pip install torch torchvision</code> and download the model weights.
Dominant colour extraction (K-Means)	Finds primary colours by clustering pixel values; widely used in colour palette extractors ⁷ ⁸ .	<code>pip install scikit-learn opencv-python</code>
Tesseract OCR (with Python bindings)	Extracts embedded text from images with an LSTM-based OCR engine ⁹ .	Install Tesseract via OS package manager (e.g. <code>sudo apt install tesseract-ocr</code>) and the Python wrapper <code>pip install pytesseract</code>
Scene classifier (Places365 or similar)	Determines the background type (indoor, outdoor, street, etc.).	<code>pip install places365-torch</code> or download a pre-trained Places365 model.
Large language model	Combines all extracted attributes into a coherent description. Use a local LLM (e.g. <code>llama.cpp</code> , <code>mistral</code>) or an API-based model.	Install or configure according to your chosen framework.

Pipeline Logic

1. **Initial caption:** Pass the image to a general vision-language model (e.g. GPT-4-V or another VLM) to get a high-level caption. This provides context for the agent and can be useful when later summarising the detailed output.
2. **Run YOLO object detection** on the image. YOLO identifies and classifies objects in a single pass ¹. The list of detected objects (person, animal, food, etc.) and their bounding boxes guides the subsequent steps.
3. **Decide which modules to run** based on the detected objects:
4. If a **person** is detected, run **MediaPipe Pose Landmarker** to extract 33 body landmarks ³ and compute orientation (head tilt, arm extension, etc.). Use this to describe posture and gestures.
5. If a **face** is visible, run **MediaPipe Face Landmarker** to obtain 468 face landmarks ⁴. Analyse eye openness, eyebrow raise, smile curvature, beard/moustache region and head pose.
6. If **clothing** (e.g. shirt, dress, jacket) is detected, isolate the region using YOLO boxes or segmentation masks. Use **MediaPipe Selfie Segmentation** to obtain hair, skin and clothes masks ⁵, then apply the garment classifier to predict category, style attributes and patterns ⁶.

7. For any **segmented region**, run K-Means to find the dominant colours ⁷ and convert them into human-readable names.
8. If objects such as **signs, books, screens or clothing tags** are detected, run **Tesseract OCR** on those cropped regions to extract text ⁹.
9. Regardless of the objects, optionally run a **scene classifier** to categorize the background (e.g. "living room", "street"). Use this when it adds relevant context.
10. **Assemble structured data.** For each object, record:
 11. Label and bounding box
 12. Pose/face attributes (if person)
 13. Clothing category and style attributes (if clothing)
 14. Colour palette
 15. Extracted text (if any)
 16. Scene label (if determined)
 17. **Generate final description.** Provide the structured data and the initial VLM caption to your language model with a prompt such as:

"Using the following structured attributes and high-level caption, write a detailed and precise description of the image. Mention all people with their poses, facial expressions and clothing. List all objects with their colours and relative positions. Mention any visible text. Describe the background. Be explicit and avoid hallucinating unseen details."

The language model then outputs a detailed paragraph.

Sample Agent Prompt (OpenClaw / Claude)

Below is an example high-level instruction you could provide to your autonomous agent, encoded in a `.md` configuration file for a tool like OpenClaw or Claude. It describes the pipeline and instructs the agent to install tools, run detections, and assemble descriptions:

```
# Description
Create an agent that produces extremely detailed descriptions of input images by combining object detection, pose estimation, facial analysis, clothing classification, segmentation, colour extraction, OCR and scene classification. The agent must decide which modules to run based on what objects are detected.

# Setup / Installation
1. Install dependencies:
  - `pip install ultralytics mediapipe scikit-learn opencv-python torch torchvision`
  - Install Tesseract OCR via the system package manager (e.g. `apt-get install tesseract-ocr`) and Python wrapper `pip install pytesseract`.
  - Download a pre-trained DeepFashion clothing classifier and a Places365 scene classifier.

# Workflow
1. When an image is received:
```

```

    a. Pass the image through a vision-language model to obtain a short initial caption.
    b. Run YOLO object detection; store all detected objects and bounding boxes
    1 .
    c. If "person" is detected:
        - Run MediaPipe Pose Landmarker to obtain 33 body landmarks 3 .
        - Run MediaPipe Face Landmarker to obtain 468 face landmarks 4 .
    d. If clothing items are detected:
        - Run MediaPipe Selfie Segmentation to get masks 5 .
        -
Run the garment classifier on the clothes region to predict category and style attributes 6 .
    - Extract dominant colours using K-Means 7 .
    e. For regions that likely contain text, run Tesseract OCR 9 .
    f. Optionally, run a scene classifier to label the background.
2. Compile the structured attributes and feed them along with the initial caption into a language model to produce a detailed description.

```

Output

Return a JSON object containing the structured attributes and the final description.

This template can be adapted to the agent framework you choose. It instructs the agent to install necessary tools, perform detection and analysis, decide which modules to use, and produce a detailed description based on the extracted information.

1 2 YOLO Object Detection Explained: A Beginner's Guide | Encord

<https://encord.com/blog/yolo-object-detection-guide/>

3 Pose landmark detection guide | Google AI Edge | Google AI for Developers

https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker

4 layout: forward target: https://developers.google.com/mediapipe/solutions/vision/face_landmarker/

title: Face Mesh parent: MediaPipe Legacy Solutions nav_order: 2 — MediaPipe v0.7.5 documentation

https://mediapipe.readthedocs.io/en/latest/solutions/face_mesh.html

5 Image segmentation guide | Google AI Edge | Google AI for Developers

https://ai.google.dev/edge/mediapipe/solutions/vision/image_segmenter

6 DeepFashion Dataset: Visual clothing recognition | Innovatiana

<https://www.innovatiana.com/en/datasets/deepfashion>

7 8 How K-Means Clustering Works (Explained by Extracting Colors from Images) - DEV Community

<https://dev.to/didof/how-k-means-clustering-works-explained-by-extracting-colors-from-images-pmi>

9 Tesseract OCR — The World's Best Open Source OCR Engine

<https://tesseractocr.org/>